Data Structures (ETCCDS202)

Unit–3 Assignment: Sorting Performance Analyzer (SPA)

Course Details

Name of the School: School of Engineering & Technology

Program: B.Tech CSE (AI ML)

Course Title: Data Structures

Course Code: ETCCDS202

Unit Title: Sorting Algorithms

Student Name: Ankit Kumar Mishra

Roll No.: 2501730079

Section: A

Analysis and Comparison (Report)

1. Table of Recorded Execution Times

(Note: Values are approximate and may vary slightly on different runs)

Input Type	Size	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
Random	1000	20-40	3-6	2-5
Random	5000	500-800	10-20	8-15
Random	10000	2000-3000	25-40	20-35
Sorted	1000	1-5	3-6	15-30
Sorted	5000	5-15	10-20	100-200
Sorted	10000	10-25	25-40	300-600
Reverse Sorted	1000	40-60	3-6	20-40
Reverse Sorted	5000	800-1200	10-20	120-250
Reverse Sorted	10000	3000-4000	25-40	400-700

2. Comparison Based on Input Type

(a) Random Data

For random data, insertion sort takes more time because it has to compare and shift elements many times.

Merge sort and quick sort are faster. Quick sort performs slightly better because it works in-place and avoids extra memory.

So overall, for random input, quick sort and merge sort are better than insertion sort.

(b) Sorted Data

For already sorted data, insertion sort performs very fast because elements are already in correct order. Only minimal comparisons are needed.

Merge sort takes almost same time because it always divides and merges regardless of input.

Quick sort becomes slower here because the pivot is chosen as the last element, which leads to unbalanced partitions.

(c) Reverse Sorted Data

For reverse sorted data, insertion sort performs the worst because every element needs to be shifted completely.

Merge sort again performs consistently with almost same time.

Quick sort also becomes slower due to poor pivot selection, similar to sorted case.

3. $O(n^2)$ vs $O(n \log n)$

From the results, it is clear that:

- Insertion sort follows **$O(n^2)$**, so its time increases very quickly as input size increases.
- Merge sort follows **$O(n \log n)$**, so it performs efficiently even for large inputs.
- Quick sort behaves like **$O(n \log n)$** for random data, but in worst cases it behaves like **$O(n^2)$**.

Thus, the practical results match the theoretical time complexities.

4. Special Note for Quick Sort

In this implementation, the **last element is used as pivot**.

- This works well for random data.
- But for sorted and reverse sorted data, it leads to worst-case behavior.

In such cases:

- Partitions become unbalanced
- Recursion depth increases
- Performance becomes slow ($O(n^2)$)

This is why quick sort takes more time in sorted and reverse sorted inputs in my results.

5. Stability and Memory Notes

- **Insertion Sort:**
  Stable and in-place
  (Does not use extra memory and maintains order of equal elements)

- **Merge Sort:**
  Stable but not in-place
  (Uses extra memory for merging process)
- **Quick Sort:**
  In-place but not stable
  (Relative order of equal elements may change)

Conclusion

From this assignment, I learned that different sorting algorithms behave differently depending on input type.

Insertion sort is good for small or already sorted data, merge sort is consistent and reliable, and quick sort is fast but depends on pivot selection.

So, choosing the right algorithm depends on the situation.